How to run

- `emcf2_with_cuts.py` - the EMCF-2 formulation with the paper's three cardinality cuts (Prop 1, Prop 2, CMI), found and added automatically.
- `lp_cglp_gurobi.py` - my own lift-and-project cuts on the bidirected (BC) model.
- `core.py` - shared helper (reads the `.stp` file, builds the graph). Both scripts need it in the same folder.

What you need

- Python 3 with `gurobipy` (academic licence), `numpy`, `scipy`, and `networkx`.
- The SteinLib instance files (`.stp`), e.g. `i080-305.stp`. Put them where you can point to them.

Install the Python packages if needed:

```
pip install gurobipy numpy scipy networkx
```

Keep `core.py` in the same folder as the two scripts.

1. The cardinality cuts (from paper) - `emcf2_with_cuts.py`

Run it with the instance file and the known optimum:

```
python emcf2_with_cuts.py i080-305.stp 5932
```

The optimum (the second number) is used only as a check: the script adds the paper's cuts one at a time and keeps a cut only if the bound stays at or below the optimum, dropping it otherwise. It never uses the optimum to build a cut.

It prints the bound each round and a final line like:

```
FINAL (valid, verified <= opt): 5888.500 -> 5932.000   closure 100.0%
```

The known optima for the instances:

Instance	Optimum	Instance	Optimum
i080-044	1366	i080-235	4487
i080-111	2051	i080-305	5932
i080-143	1767	i080-331	5226
i080-212	3677	i080-332	5362
i080-213	3678	i080-343	4246
i080-214	3734	i080-344	4310
i080-215	3681	i160-033	2101
		i160-112	2924

You can also run it without the optimum, but then there is no check and the bound is not guaranteed to be valid, so it is best to always pass the optimum.

Useful options: `--nocard`, `--noext2`, `--nomatch` turn off Prop 1, Prop 2, or CMI.

2. The lift-and-project cuts (my method) - `lp_cglp_gurobi.py`

This one needs no optimum - the cuts are valid by construction, so the bound can never go above the true answer.

Just the cut loop:

```
python lp_cglp_gurobi.py i080-305.stp
```

To also solve the integer model at the end and print how much of the gap was closed (and check the bound never went above the optimum):

```
python lp_cglp_gurobi.py i080-305.stp --ip
```

Quick sanity check that everything is set up right:

```
python lp_cglp_gurobi.py i080-235.stp --ip
```

should end with `gap closed: 100.0%` and `VALIDITY: OK` .

Useful options: `--rounds N` (how many rounds), `--budget SECONDS` (stop the cut loop after this long - some instances are slow), `--ip-only` (just time the Gurobi integer solve).